Pandas

Creating dataframes

One common way would be to read a csv file. Here, we create the file first.

In […

```
import pandas as pd
```

In […

```
s = '''
A,B,C
√2,1,1.4142
phi,4,1.6180
e,9,2.7183
pi,16,3.1415
'''

fn = 'tmp'
with open(fn,'w') as fh:  fh.write(s)
df = pd.read_csv(fn)
df
```

Out[…

	A	B	C
0	√2	1	1.4142
1	phi	4	1.6180
2	e	9	2.7183
3	pi	16	3.1415

One can also read from a dictionary.

In […

```
D = {'fips':['16','41','53'],'abbrev':['ID','OR','W
df2 = pd.DataFrame(D)
df2
```

Out[…

	fips	abbrev
0	16	ID
1	41	OR
2	53	WA

In the first example, we could have used the `io` module to treat the string as a file-like object:

In […

```
import io
df3 = pd.read_csv(io.StringIO(s), sep=',')
df3
```

Out[…

	A	B	C
0	√2	1	1.4142
1	phi	4	1.6180
2	e	9	2.7183
3	pi	16	3.1415

Assignment does **not** make a copy.

In […

```
df4 = df
print(id(df),id(df4))
```

```
4547995744 4547995744
```

In […

```
df4 = df.drop('B',axis=1)
```

In […

```
print(id(df),id(df4))
```

```
4547995744 4547999344
```

Pandas protects us by returning the result from `drop` as a new dataframe.

In […
```
df4
```

Out[…

	A	C
0	√2	1.4142
1	phi	1.6180
2	e	2.7183
3	pi	3.1415

Of course, if you really want to change the original:

In […
```
df4 = df
df.drop('B',axis=1,inplace=True)
```

In […
```
print(id(df),id(df4))
```

```
4547995744 4547995744
```

So then naturally

In […
```
df4.columns
```

Out[…
```
Index(['A', 'C'], dtype='object')
```

Working with columns

To select rows from a dataframe, first constructing a list of booleans. That list is formed by analyzing the values in one or more columns of the dataframe.

In […
```
import io
df = pd.read_csv(io.StringIO(s), sep=',')
df
```

Out[…

	A	B	C
0	√2	1	1.4142
1	phi	4	1.6180
2	e	9	2.7183
3	pi	16	3.1415

A plain old list will do.

In […

```
L = [True,False,True,False]
df[L]
```

Out[…

	A	B	C
0	√2	1	1.4142
2	e	9	2.7183

Normally the way to obtain a list of booleans is by matching the values in one or more columns against some filter. It's usually done in one step but we'll break it up by first making a selector.

We use a special pandas method `isin`.

In […

```
sel = df['A'].isin(list('abcde'))
sel
```

Out[…

```
0    False
1    False
2     True
3    False
Name: A, dtype: bool
```

In […

```
print(type(sel))
```

```
<class 'pandas.core.series.Series'>
```

In […
```
df[sel]
```

Out[…

	A	B	C
2	e	9	2.7183

Another way that works is logical OR and AND: `|`, `&`

In […
```
df[(df['A'] == 'pi') | (df['B'] == 9)]
```

Out[…

	A	B	C
2	e	9	2.7183
3	pi	16	3.1415

For complex searchs use `apply`, usually with a `lambda` expression, although it can also be a named function.

In […
```
import numpy as np

def f(row):
    v = row['B']
    def f():
        return v in range(10)
    coin_flip = np.random.choice([0,1])
    if coin_flip:
        return f()
    return True

sel = df.apply(f,axis=1)
```

In […
```
sel
```

Out[…
```
0    True
1    True
2    True
3    True
dtype: bool
```

Double brackets with column names returns those columns in the specified order.

In […
```
df1 = df[['C','B']]
df1
```

Out[…

	C	B
0	1.4142	1
1	1.6180	4
2	2.7183	9
3	3.1415	16

Adding a new column:

In […
```
df1['F'] = [1,2,3,4]
df1
```

Out[…

	C	B	F
0	1.4142	1	1
1	1.6180	4	2
2	2.7183	9	3
3	3.1415	16	4

In […
```
a = df.to_numpy()
a
```

Out[…
```
array([['√2', 1, 1.4142],
       ['phi', 4, 1.618],
       ['e', 9, 2.7183],
       ['pi', 16, 3.1415]], dtype=object)
```

In […
```
a[:,2]
```

Out[…
```
array([1.4142, 1.618, 2.7183, 3.1415], dtype=objec
t)
```

In […
```
[float(n) for n in a[:,2]]
```

Out[…
```
[1.4142, 1.618, 2.7183, 3.1415]
```

Working with rows

In […
```
import pandas as pd
import numpy as np
np.random.seed(153)

a = np.random.randint(0,10,20)
a.shape = (5,4)
df = pd.DataFrame(a,columns=list('ABCD'))
df
```

Out[…

	A	B	C	D
0	0	8	7	1
1	7	9	6	3
2	6	6	0	2
3	4	2	6	9
4	4	8	2	1

`loc` and `iloc`

In […
```
df.loc[:,'C']
```

Out[…
```
0    7
1    6
2    0
3    6
4    2
Name: C, dtype: int64
```

In […
```
df.loc[1] # 1 is a label
```

Out[…
```
A    7
B    9
C    6
D    3
Name: 1, dtype: int64
```

In […
```
df.iloc[1] # 1 is an integer index
```

Out[…
```
A    7
B    9
C    6
D    3
Name: 1, dtype: int64
```

In […
```
df = df.drop(2)
df
```

Out[…

	A	B	C	D
0	0	8	7	1
1	7	9	6	3
3	4	2	6	9
4	4	8	2	1

In […
```
df.loc[3]
```

Out[…
```
A    4
B    2
C    6
D    9
Name: 3, dtype: int64
```

In […
```
df.iloc[3]
```

Out[…
```
A    4
B    8
C    2
D    1
Name: 4, dtype: int64
```

In […
```
df.iloc[[0,3]][['D','A']]
```

Out[…

	D	A
0	1	0
4	1	4

Iterration

In […
```
for i,r in df.iterrows():
    print(r['B'])
```

```
8
9
2
8
```

Another way to access a single value:

In […
```
df.at[1,'C']
```

Out[…
```
np.int64(6)
```

In […